Chapter-2

Possible Questions

2.1 Reading and Writing Structured/Unstructured Data (CSV, JSON, Excel, Text)

1. How do you read a CSV file in Python using pandas? Provide an example.
2. How can you write a DataFrame to an Excel file?
3. Explain the difference between reading a text file with read() vs readlines().
4. How do you read a JSON file into a Python dictionary?
5. Write Python code to append data to an existing CSV file.
6. How can you handle missing values when reading CSV or Excel files?
7. What is the difference between json.load() and json.loads()?
8. How do you read only specific columns from a CSV file using pandas?
9. How can you write a list of dictionaries to a JSON file in Python?
10. Explain the difference between structured and unstructured data with examples.

2.2 Database Access with Relational and Non-Relational Databases

1. How do you connect to a SQLite database in Python and execute a simple query?
2. Explain the difference between relational and non-relational databases.
3. Write Python code to insert multiple records into a MySQL/PostgreSQL table.
4. How can you query MongoDB using pymongo in Python?
5. What is the difference between find() and find_one() in MongoDB?
6. How do you handle transactions in relational databases using Python?
7. Explain the difference between SQL and NoSQL databases with examples.
8. How do you delete a table or collection in Python?
9. What is connection pooling and why is it useful for database access?
10. How can you fetch data in chunks from a database to handle large datasets?

2.3 Accessing and Processing Data from APIs (REST, SOAP)

1. What is the difference between REST and SOAP APIs?
2. Write Python code to make a GET request to a REST API using requests.
3. How do you send JSON data in a POST request with Python?
4. What are common HTTP response codes and their meanings (200, 404, 500)?
5. How can you handle authentication (API keys, OAuth) in API requests?
6. How do you parse JSON responses from an API in Python?
7. Explain the difference between synchronous and asynchronous API calls.
8. Write Python code to handle API rate limiting gracefully.
9. How can you test an API endpoint using Python before integrating it?
10. Explain how SOAP requests differ from REST in terms of message format.

2.4 Web Scraping Using requests and BeautifulSoup

1. Write Python code to fetch a webpage using requests and parse it with BeautifulSoup.

2. How do you find all links (<a> tags) on a webpage using BeautifulSoup?
3. Explain the difference between .find() and .find_all() in BeautifulSoup.
4. How can you extract text content from HTML elements?
5. How do you handle pagination when scraping multiple pages?
6. What are some ethical considerations and rules for web scraping?
7. How can you scrape data from a table on a webpage?
8. Explain the difference between scraping static and dynamic content.
9. How can you handle request headers and user agents to avoid blocking?
10. Write code to save scraped data into a CSV file.

2.5 Handling Large Datasets with Chunking and Lazy Evaluation

1. How can you read a large CSV file in chunks using pandas?
2. Explain lazy evaluation and how it differs from eager evaluation in Python.
3. How do generators help in processing large datasets efficiently?
4. Write a generator function to read a large text file line by line.
5. How do you process a large JSON file without loading it entirely into memory?
6. Explain the difference between iterrows() and itertuples() in pandas for large datasets.
7. How can Dask be used for handling large datasets in Python?
8. What is memory mapping (mmap) and when would you use it?
9. How can you combine chunked reading with data transformation to save memory?
10. Write Python code to sum values in a large CSV file using chunking.